ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 29

Rutas y controladores

- Rutas básicas
- Rutas con parámetros
- Rutas con nombre
- Grupos de rutas
- Controladores
- La petición: el objeto Request
- Controladores de recurso
- Route model binding
- Rutas de la API y de la consola



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

En la clase 24 construimos un router capaz de entender `/cursos/{id}`, y en la 25 le pusimos controladores. Hoy vemos cómo lo hace Laravel: el mismo concepto, pero con nombres de ruta, grupos, restricciones, controladores de recurso que crean siete rutas de golpe y un truco precioso llamado *route model binding*.

Al terminar, la academia tendrá portada, página de contacto, listado de cursos con buscador y una pequeña API. Los datos todavía son provisionales: viven en un fichero de configuración hasta que lleguemos a la base de datos en la clase 31.

•
👌 Cómo vamos a trabajar



1
📘 Rutas básicas



2
👍 Rutas con parámetros



3
🚀 Rutas con nombre



4
🔍 Grupos de rutas



5
📘 Controladores



6
🤔 La petición: el objeto Request



7
😮 Controladores de recurso



8
🚀 Route model binding



9
👌 Rutas de la API y de la consola



10
🚀 Ejercicios



11
👋 Lo que tenéis que llevaros de hoy





CLASE 29

👌 Cómo vamos a trabajar

Seguimos con el proyecto `academia-laravel` de la clase 28. Antes de copiar los ejemplos, ejecutad en la carpeta del proyecto:

terminal — php-paso-a-paso/clase29 — Visual Studio Code

terminal

```
1 php artisan install:api
```

Este comando instala Sanctum (lo usaremos en la clase 36), crea el fichero `routes/api.php` y una migración. Cuando os pregunte si queréis ejecutar las migraciones, decid que sí.

Después copiad el contenido de `ejemplos/` encima del proyecto:

Esquema

```
1 config/academia.php                        ← NUEVO: datos provisionales de los cursos
2 app/Http/Controllers/PaginaController.php  ← NUEVO
3 app/Http/Controllers/CursoController.php   ← NUEVO
4 resources/views/paginas/                   ← NUEVO: inicio, contacto y aviso legal
5 resources/views/cursos/                    ← NUEVO: listado y ficha
6 routes/web.php, api.php y console.php      ← ampliados
```



💡 ¿Habéis visto que las vistas repiten la cabecera una y otra vez? Es a propósito: en la clase 30 lo arreglamos con los *layouts* de Blade.



CLASE 29 · PUNTO 1

📘 Rutas básicas

Una ruta une un **verbo HTTP** y una **dirección** con el código que la atiende [1]:

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
Route::get('/hola', function () {
    return '¡Hola desde Laravel! 👋';
});
```

Hay un método por cada verbo, igual que en nuestro router:

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
Route::get($uri, $accion);       // leer
Route::post($uri, $accion);      // crear
Route::put($uri, $accion);       // actualizar
Route::patch($uri, $accion);     // actualizar una parte
Route::delete($uri, $accion);    // borrar
Route::match(['get', 'post'], $uri, $accion);   // varios verbos
```

Y dos atajos muy cómodos:

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
// Redirige (con un 302) sin escribir ninguna función
Route::redirect('/cursos-online', '/cursos');

// Muestra una vista que no necesita datos ni controlador
Route::view('/aviso-legal', 'paginas.aviso-legal')->name('aviso-legal');
```

Fijaos en el nombre de la vista: `paginas.aviso-legal` es el fichero `resources/views/paginas/aviso-legal.blade.php`. **El punto separa carpetas.**



🔎 `php artisan route:list` muestra todas las rutas de la aplicación. Usadlo cada vez que dudéis de qué dirección atiende qué. Con `--except-vendor` oculta las que añaden los paquetes.



CLASE 29 · PUNTO 2

👍 Rutas con parámetros

Las partes variables de la dirección van entre llaves, y llegan a la función **por el nombre**:

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
// Obligatorio: /hola/Ana
Route::get('/hola/{nombre}', function (string $nombre) {
    return "¡Hola, {$nombre}! 👋";
});

// Opcional (con ?): /bienvenida  o  /bienvenida/Ana
Route::get('/bienvenida/{nombre?}', function (?string $nombre = null) {
    return '¡Bienvenido, ' . ($nombre ?? 'invitado') . '!';
});
```

Restricciones

¿Y si alguien escribe `/edad/abc`? Le decimos a la ruta qué forma debe tener el parámetro. Si no la cumple, Laravel responde con un **404** sin llegar a ejecutar nuestro código:

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
Route::get('/edad/{edad}', function (int $edad) {
    return $edad >= 18 ? 'Eres mayor de edad' : 'Eres menor de edad';
})->whereNumber('edad');
```

Método	Acepta
`whereNumber('id')`	Solo dígitos
`whereAlpha('nombre')`	Solo letras
`whereAlphaNumeric('codigo')`	Letras y dígitos
`whereIn('categoria', ['programacion', 'diseno'])`	Solo esos valores
`where('slug', '[a-z0-9-]+')`	Lo que diga la expresión regular

¿Os acordáis de la expresión regular que montábamos a mano en el router de la clase 24 para convertir `{id}` en `([^/]+)`? Esto es lo mismo, pero ya hecho.



Esta es una de las costumbres que más os van a ahorrar disgustos. A cada ruta le damos un **nombre**:

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
Route::get('/contacto', [PaginaController::class, 'contacto'])->name('contacto');
```

Y en las vistas y controladores **nunca** escribimos la dirección a mano: la generamos con `route()`:

plantilla.php — php-paso-a-paso/clase29 — Visual Studio Code

plantilla.php

```blade
<a href="{{ route('contacto') }}">Contacto</a>
<a href="{{ route('cursos.show', $curso['id']) }}">{{ $curso['titulo'] }}</a>
```

¿La ventaja? Si mañana el cliente quiere que `/contacto` pase a ser `/contactanos`, cambiamos **una línea** de `routes/web.php` y todos los enlaces se actualizan solos.

Para redirigir a una ruta con nombre desde un controlador:

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
return redirect()->route('cursos.index');
```



CLASE 29 · PUNTO 4

🔍 Grupos de rutas

Cuando varias rutas comparten algo (el principio de la dirección, el principio del nombre, un middleware...), las agrupamos:

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
Route::prefix('admin')->name('admin.')->group(function () {
    Route::get('/', function () {
        return 'Panel de administración';
    })->name('panel');          // GET /admin          → admin.panel

    Route::get('/cursos', function () {
        return 'Gestión de cursos';
    })->name('cursos');         // GET /admin/cursos   → admin.cursos
});
```

En la clase 34 le añadiremos `->middleware('auth')` para que solo entren los usuarios identificados: exactamente el `grupo()` con middleware que hicimos en la clase 26.



CLASE 29 · PUNTO 5

📘 Controladores

Escribir todo el código dentro de `routes/web.php` está bien para probar, pero en cuanto crece se vuelve inmanejable. Igual que en nuestro MVC, el código va en **controladores** [2]. Los crea Artisan:

terminal — php-paso-a-paso/clase29 — Visual Studio Code

terminal

```
php artisan make:controller PaginaController
```

Esto genera `app/Http/Controllers/PaginaController.php`. Le añadimos dos métodos (ver fichero):

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
<?php

namespace App\Http\Controllers;

use Illuminate\View\View;

class PaginaController extends Controller
{
    public function inicio(): View
    {
        return view('paginas.inicio', [
            'totalCursos' => count(config('academia.cursos')),
        ]);
    }

    public function contacto(): View
    {
        return view('paginas.contacto', [
            'telefono' => config('academia.telefono'),
            'email' => config('academia.email'),
        ]);
    }
}
```

Y en las rutas indicamos **clase y método** con un array:

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
use App\Http\Controllers\PaginaController;

Route::get('/', [PaginaController::class, 'inicio'])->name('inicio');
Route::get('/contacto', [PaginaController::class, 'contacto'])->name('contacto');
```

`PaginaController::class` es el nombre completo de la clase (`App\Http\Controllers\PaginaController`) como texto. Así, si nos equivocamos al escribirlo, VS Code nos avisa.

Los datos provisionales

Hasta la clase 31 no tenemos base de datos, así que los cursos viven en un fichero de configuración propio, `config/academia.php`. Cualquier fichero que creemos en `config/` se lee con `config()`:

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
return [
    'telefono' => '942 000 000',
    'email' => 'info@academia-alpe.test',

    'cursos' => [
        1 => ['id' => 1, 'titulo' => 'PHP desde cero', 'categoria' => 'programacion',
'horas' => 60, 'precio' => 250.0],
        // ...
    ],
];
```

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
config('academia.telefono');   // "942 000 000"
config('academia.cursos.3');   // el array del curso 3, o null si no existe
```



💡 **Convención de nombres:** las clases que genera Laravel llevan el sufijo en inglés (`CursoController`, `CursoRequest`, `CursoPolicy`…), porque así lo hacen Artisan y la documentación. Nosotros ponemos el nombre en español y dejamos el sufijo como lo espera el *framework*.

Controladores de una sola acción

Si un controlador solo hace una cosa, se le puede poner un único método `__invoke()` y crearlo con `php artisan make:controller BuscarController --invokable`. En la ruta basta con la clase: `Route::get('/buscar', BuscarController::class)`.



CLASE 29 · PUNTO 6

🤔 La petición: el objeto Request

Nuestra clase `Peticion` de la clase 24 tiene su equivalente: `Illuminate\Http\Request`. Para recibirla, basta con **pedirla** como parámetro del método; Laravel nos la da (es el contenedor de dependencias de la clase 26 trabajando) [3]:

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
use Illuminate\Http\Request;

// GET /cursos  y  GET /cursos?buscar=php
public function index(Request $request): View
{
    $buscar = $request->query('buscar', '');
    $cursos = config('academia.cursos');

    if ($buscar !== '') {
        $cursos = array_filter(
            $cursos,
            fn (array $curso) => str_contains(mb_strtolower($curso['titulo']),
mb_strtolower($buscar))
        );
    }

    return view('cursos.index', [
        'cursos' => $cursos,
        'buscar' => $buscar,
    ]);
}
```

Método	Qué devuelve	Equivale a…
`$request->query('buscar', '')`	Un dato de la URL (`?buscar=`)	`$_GET['buscar'] ?? ''`
`$request->input('nombre')`	Un dato del formulario (o de la URL)	`$_POST['nombre'] ?? null`
`$request->all()`	Todos los datos	`$_GET + $_POST`
`$request->has('email')`	Si viene ese dato	`isset(...)`
`$request->method()`	El verbo: `GET` , `POST` …	`$_SERVER['REQUEST_METHOD']`
`$request->path()`	La ruta: `cursos/3`	
`$request->ip()`	La IP del visitante	`$_SERVER['REMOTE_ADDR']`

Si una ruta tiene parámetros **y** queremos la petición, se ponen las dos cosas: Laravel distingue por el tipo.

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
public function show(Request $request, int $curso) { ... }
```

Responder con un error

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
public function show(int $curso): View
{
    $datos = config("academia.cursos.$curso");

    if ($datos === null) {
        abort(404);
    }

    return view('cursos.show', ['curso' => $datos]);
}
```

`abort(404)` lanza una excepción HTTP que Laravel convierte en la página de error correspondiente. Es nuestro `HttpException` de la clase 25.



Un CRUD siempre necesita las mismas siete rutas. Laravel las crea con una sola línea [2]:

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
Route::resource('cursos', CursoController::class);
```

Verbo	Dirección	Método	Nombre	Para qué
GET	`/cursos`	`index`	`cursos.index`	Listado
GET	`/cursos/create`	`create`	`cursos.create`	Formulario de alta
POST	`/cursos`	`store`	`cursos.store`	Guardar el nuevo
GET	`/cursos/{curso}`	`show`	`cursos.show`	Ficha de uno
GET	`/cursos/{curso}/edit`	`edit`	`cursos.edit`	Formulario de edición
PUT/PATCH	`/cursos/{curso}`	`update`	`cursos.update`	Guardar cambios
DELETE	`/cursos/{curso}`	`destroy`	`cursos.destroy`	Borrar

Esta tabla os la vais a saber de memoria. Es la **convención REST** de la clase 27 aplicada a una web.

El controlador con los siete métodos vacíos lo genera Artisan:

terminal — php-paso-a-paso/clase29 — Visual Studio Code

terminal

```
1 php artisan make:controller CursoController --resource
```

Hoy solo tenemos datos de lectura, así que limitamos las rutas a dos:

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```
1 Route::resource('cursos', CursoController::class)->only(['index', 'show']);
2 // o al revés: ->except(['destroy'])
```



⚠️ **El plural en español.** Laravel calcula el nombre del parámetro quitando la «s» final, con reglas del inglés: `Route::resource('profesores', ...)` genera `/profesores/{profesore}`. 🙃 Se arregla así: `->parameters(['profesores' => 'profesor'])`. Lo practicaréis en el ejercicio 02.



CLASE 29 · PUNTO 8

🚀 Route model binding

Pensad en cuántas veces hemos escrito esto en nuestro MVC:

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
$curso = Curso::buscar($id);
if ($curso === null) {
    $this->noEncontrado();
}
```

Laravel lo hace por nosotros. Si el parámetro de la función tiene como **tipo un modelo** y se llama **igual** que el parámetro de la ruta, Laravel busca el registro por su `id` y nos lo da ya cargado. Si no existe, responde con un 404 [4]:

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
use App\Models\User;

Route::get('/usuarios/{user}', function (User $user) {
    return "Usuario {$user->id}: {$user->name} ({$user->email})";
})->name('usuarios.show');
```

De momento solo tenemos un modelo, el de los usuarios que trae Laravel. Para probarlo, cread uno con Tinker:

Esquema

```
php artisan tinker
> App\Models\User::factory()->create(['name' => 'Lucía Pérez', 'email' =>
'lucia@example.com'])
```

Y abrid `/usuarios/1` (existe) y `/usuarios/99` (404). En la clase 32, cuando los cursos sean modelos de Eloquent, el `show()` de `CursoController` quedará así de corto:

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
public function show(Curso $curso): View
{
    return view('cursos.show', ['curso' => $curso]);
}
```



CLASE 29 · PUNTO 9

👌 Rutas de la API y de la consola

routes/api.php

Las rutas de este fichero tienen dos diferencias con las de la web [5]:

- Todas empiezan por `/api` sin que lo escribamos.
- No usan sesión ni cookies (ni, por tanto, CSRF): son para programas, como vimos en la clase 27.

Y si una ruta devuelve un **array**, Laravel lo convierte en **JSON** con su cabecera. ¡Adiós a nuestro `Respuesta::json()`!

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
// GET /api/estado
Route::get('/estado', function () {
    return [
        'aplicacion' => config('app.name'),
        'estado' => 'ok',
        'hora' => now()->toIso8601String(),
    ];
});

// GET /api/cursos
Route::get('/cursos', function () {
    return array_values(config('academia.cursos'));
});
```

Los errores de las rutas `/api/...` también salen en JSON, porque así lo configura `bootstrap/app.php`. En la clase 36 haremos una API de verdad.

routes/console.php

Los comandos de Artisan también son «rutas»: en lugar de una dirección, reciben un **nombre de comando**. Ampliamos el fichero con uno que lista los cursos y acepta una **opción** (las opciones llevan `--`):

ejemplo.php — php-paso-a-paso/clase29 — Visual Studio Code

ejemplo.php

```php
// php artisan academia:cursos   o   php artisan academia:cursos --categoria=diseno
Artisan::command('academia:cursos {--categoria=}', function () {
    $categoria = $this->option('categoria');
    $cursos = config('academia.cursos');

    if ($categoria !== null) {
        $cursos = array_filter($cursos, fn (array $curso) => $curso['categoria'] ===
$categoria);
    }

    if ($cursos === []) {
        $this->warn('No hay cursos en esa categoría.');

        return;
    }

    $this->table(['Id', 'Título', 'Categoría', 'Horas', 'Precio'], $cursos);
})->purpose('Lista los cursos de la academia');
```

En la definición	Es...	Se lee con...
`{nombre}`	Argumento obligatorio	`$this->argument('nombre')`
`{nombre?}` / `{nombre=Ana}`	Argumento opcional / con valor por defecto	`$this->argument('nombre')`
`{--categoria=}`	Opción con valor	`$this->option('categoria')`
`{--todos}`	Opción sí/no	`$this->option('todos')` (`true` o `false`)



CLASE 29 · PUNTO 10

🚀 Ejercicios

Las soluciones se copian dentro del proyecto, respetando sus carpetas.

- ☐ **01 · Cursos por categoría.** Añadid a `CursoController` el método `porCategoria()` y la ruta `GET /cursos/categoria/{categoria}` (con nombre `cursos.categoria`), que solo acepte letras minúsculas y muestre el listado de cursos de esa categoría. Si no hay ninguno, un 404. → `CursoController.php` y `routes/web.php`
- ☐ **02 · Profesores.** Añadid a `config/academia.php` una lista de profesores y cread `ProfesorController` como controlador de recurso con `index` y `show`, sus vistas y la ruta `Route::resource()`. Comprobad con `route:list` que el parámetro se llama `{profesor}` y no `{profesore}`. → `ProfesorController.php`, `config/academia.php`, `vistas` y `routes/web.php`

- ☐ **03 · Un curso en la API.** Cread la ruta `GET /api/cursos/{id}`, que devuelva en JSON el curso o un 404 con el mensaje «Curso no encontrado». → `routes/api.php`



🏆 **Reto:** añadid a `PaginaController` un método `buscar()` y una ruta `GET /buscar?q=...` que busque el texto tanto en los cursos como en los profesores y muestre los resultados en una vista con dos listas.



CLASE 29 · PUNTO 11

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
`Route::get()` …	Un verbo + una dirección + quién la atiende
`{id}` / `{id?}`	Parámetros obligatorios y opcionales; llegan por nombre
`whereNumber()` …	Restricciones: si no se cumplen, 404
`->name()` + `route()`	Nunca escribimos direcciones a mano
`Route::prefix()->group()`	Agrupar rutas que comparten prefijo, nombre o middleware
`[Controlador::class, 'metodo']`	Cómo se apunta a un controlador
`Request $request`	La petición, inyectada por Laravel
`Route::resource()`	Las 7 rutas REST de un CRUD en una línea
Route model binding	`{user}` + `User $user` = registro cargado o 404
`routes/api.php`	Prefijo `/api`, sin sesión, los arrays salen en JSON
`routes/console.php`	Comandos de Artisan con argumentos y opciones

📚 Fuentes y documentación oficial

1. Laravel: Routing
2. Laravel: Controllers
3. Laravel: HTTP Requests
4. Laravel: Route Model Binding
5. Laravel: Artisan Console (Closure Commands)